



UNIVERSITY OF ENGINEERING AND TECHNOLOGY MARDAN

<i>NAME</i>	<i>RIFAQ AJMAL</i>
<i>REG NO</i>	<i>23MDBCS435</i>
<i>SECTION</i>	<i>CS</i>
<i>SEMESTER</i>	<i>5th</i>
<i>SUBJECT</i>	<i>OS LAB</i>

Mini Project (OEL)

SELECTED : Shortest Job First (SJF) Process Scheduling Simulator

1. Introduction

An operating system is responsible for managing system resources efficiently. One of its most important tasks is **CPU scheduling**, which decides which process gets the CPU and for how long.

This project implements the **Shortest Job First (SJF) Non-Preemptive scheduling algorithm** using the C programming language. The simulator helps in understanding how processes are scheduled based on their burst time and how performance metrics such as waiting time and turnaround time are calculated.

2. Objective of the Project

The main objectives of this project are:

- To understand the working of **SJF Non-Preemptive scheduling**
- To simulate process execution in a CPU
- To calculate performance metrics:
 - Waiting Time
 - Turnaround Time
- To visualize process execution using a **text-based Gantt Chart**
- To strengthen practical knowledge of operating system concepts

3. Algorithm Used (SJF – Non-Preemptive)

Shortest Job First is a scheduling algorithm in which:

- The process with the **smallest burst time** is selected first
- Only processes that have **already arrived** are considered
- Once a process starts execution, it **runs until completion** (non-preemptive)

Steps of the Algorithm

1. Read number of processes and their arrival & burst times
2. Set current time = 0
3. Select the process with the **minimum burst time** among arrived processes
4. Execute the selected process completely
5. Calculate waiting time and turnaround time
6. Repeat until all processes are completed

4. Design and Data Structures

A **structure (struct)** is used to represent a process with the following attributes:

- Process ID (PID)
- Arrival Time
- Burst Time
- Waiting Time
- Turnaround Time
- Completion Status

This structure makes the program easy to understand and maintain.

5. Implementation Details

The simulator is implemented in **C language** using:

- Arrays of structures
- While loop for scheduling
- Conditional logic to select shortest job
- Simple calculations for performance metrics

The program also prints a **Gantt Chart** to show execution order of processes.

6. Visual Representation (Gantt Chart)

The Gantt chart is displayed in text form in the terminal:

```
| P3 | P2 | P1 |
```

This visualization helps in understanding:

- Order of execution
- CPU allocation
- Scheduling behavior of SJF

7. Performance Metrics

The following metrics are calculated:

Waiting Time (WT)

$WT = \text{Start Time} - \text{Arrival Time}$

Turnaround Time (TAT)

$TAT = \text{Completion Time} - \text{Arrival Time}$

Average Values

- Average Waiting Time
- Average Turnaround Time

These values are printed at the end of the simulation.

8. Testing and Results

The program was tested using multiple test cases with different arrival and burst times. Results showed that:

- Processes with shorter burst time finish earlier
- Average waiting time is reduced compared to FCFS
- The algorithm works correctly for non-preemptive scheduling

9. Conclusion

This project successfully demonstrates the working of the **Shortest Job First (Non-Preemptive) scheduling algorithm**.

Through this simulator, we learned:

- How CPU scheduling decisions affect performance
- How waiting and turnaround times are calculated
- The importance of burst time in scheduling

The project strengthened our understanding of operating system concepts and practical implementation using C language.

10. Future Improvements

- Add **Priority Scheduling**
- Add **Round Robin scheduling**
- Convert text visualization into GUI
- Add dynamic process arrival during execution

CODE

```
rifaq@rifaq: ~  
[1/3] Mini *  
#include <stdio.h>  
  
struct Process {  
    int pid;           // Process ID  
    int arrival;       // Arrival Time  
    int burst;         // Burst Time  
    int waiting;       // Waiting Time  
    int turnaround;    // Turnaround Time  
    int completed;     // Completion flag  
};  
  
int main() {  
    int n, time = 0, completed = 0;  
    float totalWT = 0, totalTAT = 0;  
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    struct Process p[n];  
  
    // Input process details  
    for (int i = 0; i < n; i++) {  
        p[i].pid = i + 1;  
        printf("\nProcess %d Arrival Time: ", p[i].pid);  
        scanf("%d", &p[i].arrival);  
        printf("Process %d Burst Time: ", p[i].pid);  
        scanf("%d", &p[i].burst);  
        p[i].completed = 0;  
    }  
  
    printf("\n----- GANTT CHART ----- \n");  
  
    // SJF Scheduling  
    while (completed < n) {  
        int index = -1;  
        int minBurst = 9999;  
  
        for (int i = 0; i < n; i++) {  
            if (p[i].arrival <= time && p[i].completed == 0) {  
                if (p[i].burst < minBurst) {  
                    minBurst = p[i].burst;  
                    index = i;  
                }  
            }  
        }  
  
        if (index == -1) {  
            time++; // CPU idle  
        } else {  
            time += p[index].burst;  
            p[index].completed = 1;  
            completed++;  
            totalWT += p[index].waiting;  
            totalTAT += p[index].turnaround;  
        }  
    }  
}
```

```

        printf(" P%d |", p[index].pid);

        p[index].waiting = time - p[index].arrival;
        time += p[index].burst;
        p[index].turnaround = time - p[index].arrival;
        p[index].completed = 1;
        completed++;
    }
}

// Display results
printf("\n\nPID\tAT\tBT\tWT\tTAT\n");

for (int i = 0; i < n; i++) {
    totalWT += p[i].waiting;
    totalTAT += p[i].turnaround;

    printf("P%d\t%d\t%d\t%d\t%d\n",
           p[i].pid,
           p[i].arrival,
           p[i].burst,
           p[i].waiting,
           p[i].turnaround);
}

printf("\nAverage Waiting Time = %.2f", totalWT / n);
        p[i].turnaround);
}

printf("\nAverage Waiting Time = %.2f", totalWT / n);
printf("\nAverage Turnaround Time = %.2f\n", totalTAT / n);

return 0;
}

```

OUTPUT

```
Ubuntu 64-bit - VMware Workstation 17 Player
Player
rifaq@rifaq:~$ ./sjf.c
bash: ./sjf.c: Permission denied
rifaq@rifaq:~$ gcc sjf.c -o sjf
rifaq@rifaq:~$ ./sjf
Enter number of processes: 4

Process 1 Arrival Time: 0
Process 1 Burst Time: 5

Process 2 Arrival Time: 1
Process 2 Burst Time: 3

Process 3 Arrival Time: 2
Process 3 Burst Time: 8

Process 4 Arrival Time: 3
Process 4 Burst Time: 6

----- GANTT CHART -----
| P1 | P2 | P4 | P3 |

PID   AT   BT   WT   TAT
P1     0    5    0    5
P2     1    3    4    7
P3     2    8   12   20
P4     3    6    5   11

Average Waiting Time = 5.25
Average Turnaround Time = 10.75
rifaq@rifaq:~$
```